

ANUGAT AI

Samayak · The Academic Operations Platform

Engineering Assignment

Admin Panel — Full-Stack Build

Role	Full-Stack Engineer
Module	Admin Panel — design, build & deploy from scratch
Duration	5 days
Deliverable	GitHub repository + deployed live link
Design reference	Samayak Design System (link in Section 3)
Version	v3.0 — June 2026

Confidential — for the hiring process only. Please do not share.

01 Context

Read this before you start. The why behind the product shapes every decision you'll make in the panel.

Samayak is the academic operations platform built by Anugat AI for Indian colleges and universities. University timetabling is formally an NP-hard problem — assigning faculty, rooms, subjects, and time slots without conflicts — and most institutions still do it by hand. Samayak closes that loop across four stages: Plan (AI-assisted, conflict-free scheduling), Operate (real-time view of planned vs actual), Recover (structured workflows to reschedule lost classes instead of absorbing them), and Improve (tracking delivery data so the next semester's plan gets better).

The Admin Panel is the foundation everything else stands on. It is where a university administrator sets up and manages the core data — departments, rooms, courses, and users — that every scheduling, recovery, and analytics feature depends on. We already have working interfaces for coordinators, HODs, deans, and faculty. The Admin Panel is the missing piece.

02 The Task

Design and build the Samayak Admin Panel as a working full-stack application — frontend, backend APIs, and a real database. Section 5 defines the pages and their capabilities. How you structure, lay out, and present them is yours to decide.

The most important thing we are evaluating is not whether you built all five pages — it is whether you built them well. UX is the primary lens. Design-system alignment is the second. Everything else follows.

One specific test: the Dashboard must not be a static summary screen. We will feed your system new data mid-review (Section 8) and watch how the analytics respond. Build the dashboard as a live analytical instrument, not a counts panel.

03 Design System

There is no Figma file for this assignment. Your single source of design truth is the Samayak Design System:

<https://serveranugatai-sudo.github.io/Samayak-Design-System>

Your panel must reflect the Figtree typeface, the soft-blue palette and brand gradient (#256199 to #3DA1FF), rounded geometry, and the button, card, and icon language the system defines. We are not looking for pixel-perfect cloning of a mockup that does not exist — we are looking for a panel that is unmistakably Samayak.

04 Working Data

Seed your database from the provided CSE department timetable (BIT Mesra, Spring 2026). This is your primary dataset for building and testing. From it you can derive:

- **Departments & branches** — CSE, with branches CS, AIML, MCA, and M.Tech across multiple semesters and sections (VI A-D, IV A-D, etc.)
- **Rooms** — lecture rooms (219, 220, 233A) and labs (Lab 1-7), with type and capacity
- **Courses** — course codes, names, credits, and type (lecture / lab / tutorial), scoped per branch and semester

- **Faculty** — teachers assigned to each course, mapped to their roles

Seed with all of it. The system must handle real volume and real edge cases — zero-credit subjects, shared faculty across sections, multi-type rooms. **The PDF Ingestion page (Section 5.6) will be used to load a second department's data during evaluation.**

05 Admin Pages to Build

Six pages. Build these well rather than building more. Layout, navigation, and interaction patterns are yours to design.

5.1 Dashboard / Analytics *the admin's analytical instrument — not a summary screen*

This is where we most closely evaluate the candidate's product thinking. The dashboard must present actionable analytics derived from the institution's timetable data — not static counts. The four required metrics are:

Room Utilisation %

utilisation = (slots occupied / total available slots) x 100 [per room, per day, overall]

Shows which rooms are over-relied on and which are sitting idle. Displayed per room and as a department-wide figure. Should update when new data is imported.

Probability of Finding an Empty Room — per Time Slot

$P(\text{empty} \mid \text{slot } S) = \text{free rooms in slot } S / \text{total rooms}$ [for each period I through IX]

Directly supports class recovery. A coordinator looking for a replacement slot needs to know which time periods have rooms available. The higher this probability, the safer the recovery window.

Under-Running Courses

under-running = courses where scheduled contact slots < (credit hours implied by credit count)

Flags courses that are behind their required delivery. A 4-credit lecture course implies 4 contact hours per week; if only 3 slots are scheduled, it is under-running. The dashboard should list these and quantify the gap.

Average Empty Room-Hours per Day

avg empty room-hours = mean over all rooms of (unscheduled slots x slot duration) across a working day

Quantifies idle infrastructure. High empty room-hours alongside low room utilisation signals scheduling inefficiency and wasted capacity.

In addition to these four: **the dashboard must react to live data changes.** Import a new department and the metrics should recalculate. Add a room and utilisation should shift. This reactivity is what separates a real analytical instrument from a static display.

5.2 Departments *departments & branches*

- Add a department/branch manually with a name and unique short code.
- Bulk import from Excel/CSV with per-row validation and a readable pass/fail report.
- Searchable, paginated list. Edit and delete with confirmation; deleting a department that other records depend on should warn, not silently break.

5.3 Rooms *lecture rooms & labs*

- Add and import rooms: room number, owning department, capacity, and type (classroom / lab / other).
- Capacity is required and rooms without it should be flagged. Room type shown clearly (e.g. a badge). Rooms are unique within a department.
- Search, edit, and delete. Adding or removing a room should update the analytics dashboard.

5.4 Courses / Subjects *scoped per branch & semester*

- Filter by branch and semester; the list updates to that scope and the admin always knows what scope they are editing.
- Add and import courses: code, name, credits, and type. Zero-credit subjects allowed but flagged.
- Credit count is the basis for the under-running metric — store it correctly.

5.5 Faculty & Users *the people in the system*

- Import users from Excel/CSV: name, email, role, department. Preview parsed rows before committing; let the admin handle duplicates (skip or merge) gracefully.
- Roles map to the system hierarchy (admin, coordinator, professor, HOD, dean), shown with clear visual distinction.
- Searchable, paginated list. Deletes should be recoverable rather than instantly destructive.

5.6 Timetable PDF Ingestion *upload a department PDF — the system does the rest*

- A file upload area that accepts a PDF in the BIT Mesra timetable format. Shows live processing status (queued, parsing, integrating, done).
- On completion, displays an import summary: entities created, entities matched to existing records, and rows that could not be parsed — with the reason.
- All dashboard analytics must recompute automatically once ingestion completes. The admin should see the updated numbers without a manual refresh.
- Build for the format, not for CSE. Any department's timetable in the same PDF layout must work without code changes.

Bulk import (Excel/CSV) is a shared capability across pages 5.2–5.5. Design one import experience and reuse it. The PDF ingestion page (5.6) is a distinct, dedicated flow.

06 Technical Baseline

The backend stack below is prescribed. The frontend framework is your choice. All code must be TypeScript in strict mode.

Infrastructure & Runtime

Technology	Role	Requirement
Docker + Compose	Containerises the full stack	One docker-compose up starts everything: API, workers, PostgreSQL, and Redis. No manual steps beyond copying .env.
PostgreSQL	Primary database	All entities stored with proper foreign keys, constraints, and indexes.
Prisma ORM	Database client & migrations	Type-safe queries, schema versioning, and migration history committed to the repo.
Redis	In-memory store	Powers BullMQ queues; also used for caching and rate-limit counters.
BullMQ	Async job queue	All heavy async work (bulk imports, exports, notifications) runs as BullMQ jobs so the main API thread is never blocked.

API Safety & Observability

Technology	Role	Requirement
RBAC Middleware	Role-based access control	Every route checks the caller's role (admin, coordinator, HOD, dean, faculty). Unauthorised requests return a clear 403.
Correlation IDs	End-to-end request tracing	Each request gets a unique ID that flows through all logs so any request can be traced completely.
GET /api/health	Deployment health check	Returns live status of DB, Redis, and queue connections. Must respond correctly at the submitted URL.

Absolute requirement: no mocked or hardcoded data anywhere. CRUD, search, pagination, bulk import, and all analytics must run against a live database. Make sure basic aspects such as role based access is handled.

You can use AI for writing the code but you must ensure that you don't expose vulnerabilities of a vibe coded application . Failing to ensure this will hurt your evaluation.

07 What We'll Judge

In rough order of weight:

- **User experience (important)** — intuitive flows, sensible defaults, clear feedback, graceful error and empty states, overall polish.
- **Analytics quality (important)** — whether the four metrics are computed correctly, update the moment new data is ingested via the PDF page, and are presented as genuinely useful signals — not just numbers on a screen.
- **PDF ingestion (important)** — how well the system parses and integrates an unseen department's timetable. Accuracy of extraction, handling of edge cases in the PDF, and correctness of the analytics recomputation are all evaluated.
- **Design-system alignment** — how faithfully and consistently the UI reflects Samayak (type, colour, gradient, components, geometry).

- **Functionality** — all six pages working end-to-end: CRUD, search, import, validation, safe deletes, and PDF ingestion.
- **Code quality & architecture** — sensible structure, clean APIs, type safety, decisions you can defend.

Automatic concerns: mocked/hardcoded data, a demo we cannot access, or a build that does not run will significantly hurt your evaluation.

08 Timetable PDF Ingestion Page

This is the unseen-scenario test. It is the most technically demanding page in the assignment.

Build a dedicated page in the admin panel where a reviewer can upload a department timetable in PDF — in the same format as the CSE data provided — and the system will parse, extract, and integrate it automatically.

The page must do the following:

1. **Accept a PDF upload** in the BIT Mesra timetable format. The reviewer drops the file, the system processes it asynchronously (BullMQ), and the page shows live progress feedback.
2. **Extract structured data** from the PDF: department name, branch and section identifiers, time slots and days, course codes and names, room identifiers, and faculty names — mapping each to the correct entity in the database.
3. **Integrate into the system** — departments, rooms, courses, and faculty that do not exist are created; those that already exist are matched and linked, not duplicated. The reviewer sees a clear import summary: what was created, what was matched, and what could not be parsed.
4. **Trigger analytics recalculation** — the moment ingestion completes, all four dashboard metrics (room utilisation, P(empty room) per slot, under-running courses, avg empty room-hours) must recompute to reflect the newly added department.
5. **Error handling (Bonus Point)** — If the uploaded PDF does not fully align with the expected format or is missing information, the system should degrade gracefully rather than fail entirely. This is an open-ended evaluation criterion.

Possible approach: The system ingests whatever structured data it can extract, flags what could not be parsed or was incomplete (missing room capacity, absent faculty assignments, etc.), and creates a "pending" or "incomplete" state for those entities. The admin can then upload a supplementary PDF or manually fill gaps later without re-importing the entire dataset.

Your creativity here matters. Build an ingestion flow that is robust to real-world PDF variations — tables with merged cells, inconsistent column orders, missing sections, or partial data — and demonstrate how the system handles degradation without losing what it could parse. This is how real production systems behave.

What is being evaluated: The reviewers will upload a timetable PDF for a department the system has never seen. **How well the system handles that unseen data is the core test.** Accuracy of extraction, graceful handling of parse failures, correct entity matching, and live analytics update are all judged.

Build the parser to be format-aware, not brittle. The table layout, column order, and heading styles in the CSE PDF are the contract — build to that contract and the system will work for any department's data in the same format.

The upload PDF will not be shared before submission. Build for the format, not for CSE specifically.

09 Deliverables

- **GitHub repository** — send a collaborator invite to [serveranugatai-sudo](#). One command (or a short documented sequence) should bring the full stack up locally.

- **Live demo URL** — deployed and reachable without us configuring anything. Pre-seed with CSE data and provide a demo admin login.(In the login page itself create a section use demo login)
- **README** — setup steps, environment variables, how to seed the CSE data, and a short log of key design and technical decisions (and trade-offs given the five-day window).
- **Seed data** — scripts or files that populate the database from the provided CSE dataset.

Missing or inaccessible deliverables count against you. Make it easy for us to run and review.

10 Submission & Questions

Submit to	tech@anugatai.com
Subject line	[SUBMISSION] Your Name — Admin Panel
GitHub access	Invite collaborator: serveranugatai-sudo
Questions	Email tech@anugatai.com — we aim to respond within 24 hours

Build something you would be proud to ship.
